Applied Bayesian Statistics in Animal & Biological Sciences

6) Parameter calibration of ordinary differential equation models

Henk J. van Lingen

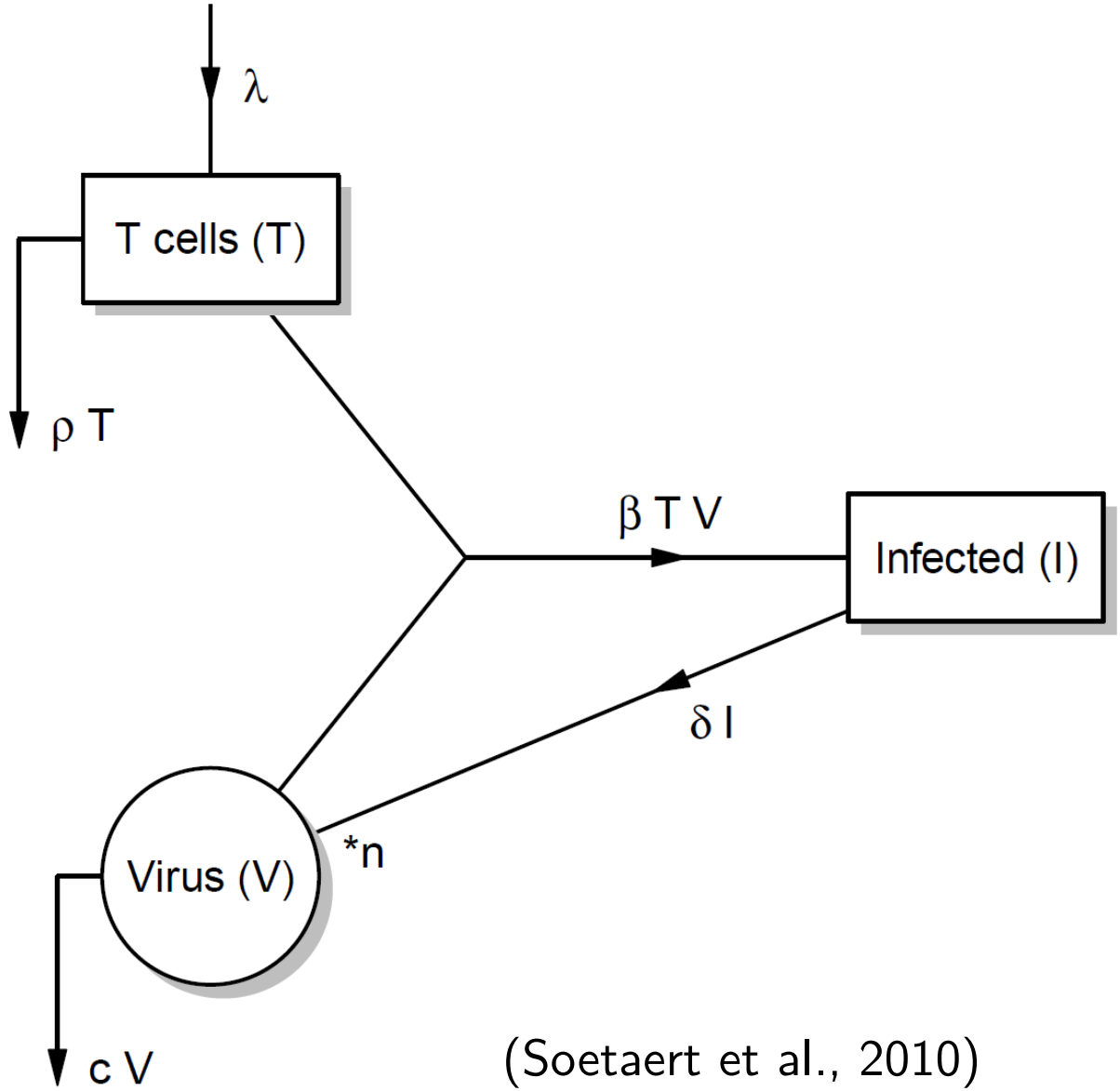
hvanling@uoguelph.ca

May 15, 2026

Today's program (focus on immunology modeling)

- HIV model
- Bayesian ordinary differential equation (ODE) model parameter calibration
- Bayesian MCMC algorithms and sampling schemes
- Posterior predictive distributions for various noise levels and priors
- SIR/SEIR model

Model of HIV virus dynamics in human blood cells



(Soetaert et al., 2010)

Model of HIV virus dynamics in human blood cells

Number of uninfected $CD4^+$ T lymphocytes (T)

$$\frac{dT}{dt} = \lambda - \rho T - \beta T V \tag{1}$$

Model of HIV virus dynamics in human blood cells

Number of uninfected $CD4^+$ T lymphocytes (T)

$$\frac{dT}{dt} = \lambda - \rho T - \beta T V \tag{1}$$

Number of infected $CD4^+$ T lymphocytes (I)

$$\frac{dI}{dt} = \beta T V - \delta I \tag{2}$$

Model of HIV virus dynamics in human blood cells

Number of uninfected $CD4^+$ T lymphocytes (T)

$$\frac{dT}{dt} = \lambda - \rho T - \beta TV \tag{1}$$

Number of infected $CD4^+$ T lymphocytes (I)

$$\frac{dI}{dt} = \beta TV - \delta I \tag{2}$$

Number of free virions (V)

$$\frac{dV}{dt} = n\delta I - cV - \beta TV \tag{3}$$

Model of HIV virus dynamics in human blood cells

Number of uninfected $CD4^+$ T lymphocytes (T)

$$\frac{dT}{dt} = \lambda - \rho T - \beta TV \tag{1}$$

Number of infected $CD4^+$ T lymphocytes (I)

$$\frac{dI}{dt} = \beta TV - \delta I \tag{2}$$

Number of free virions (V)

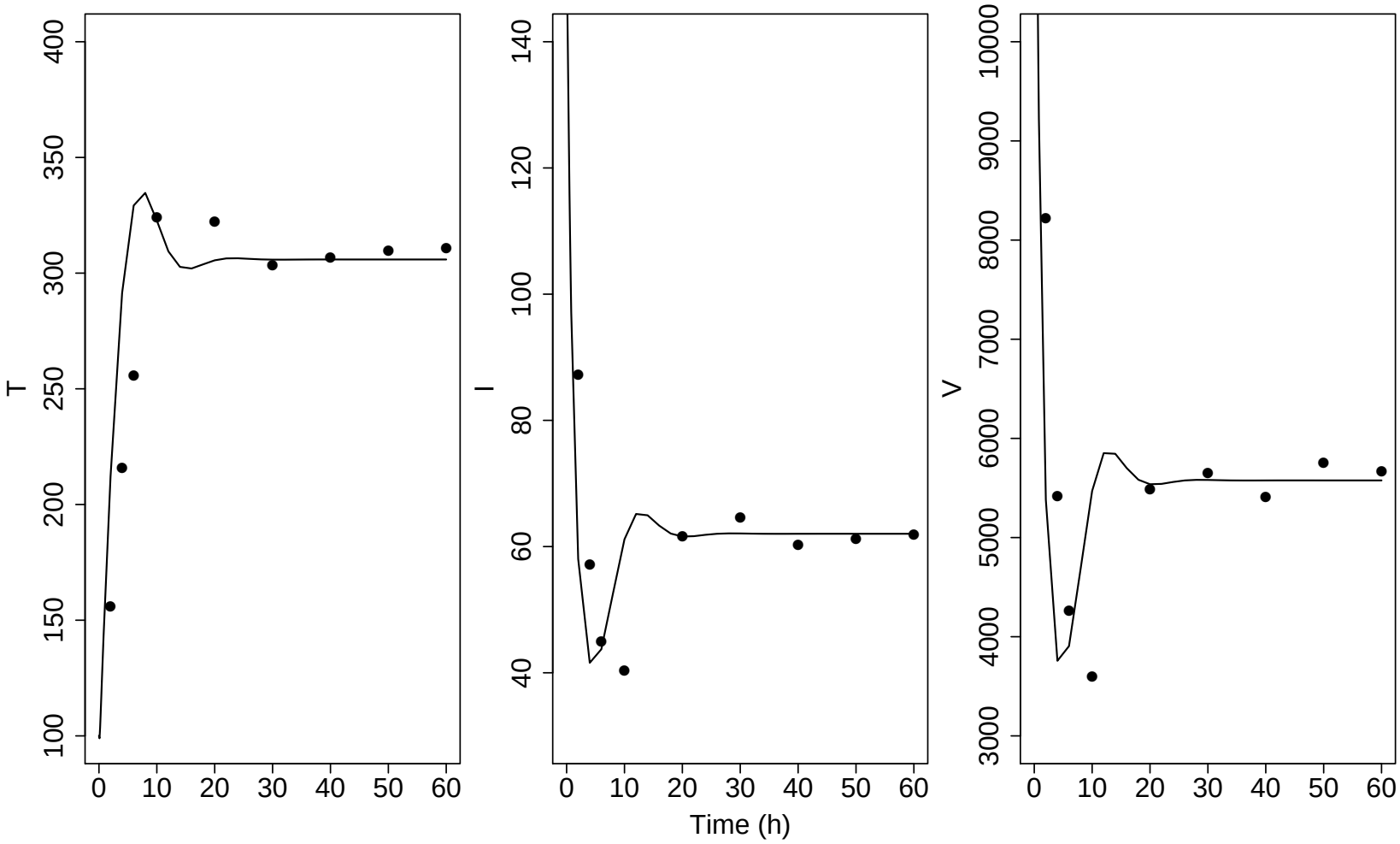
$$\frac{dV}{dt} = n\delta I - cV - \beta TV \tag{3}$$

- These 3 differential equations use 6 parameters

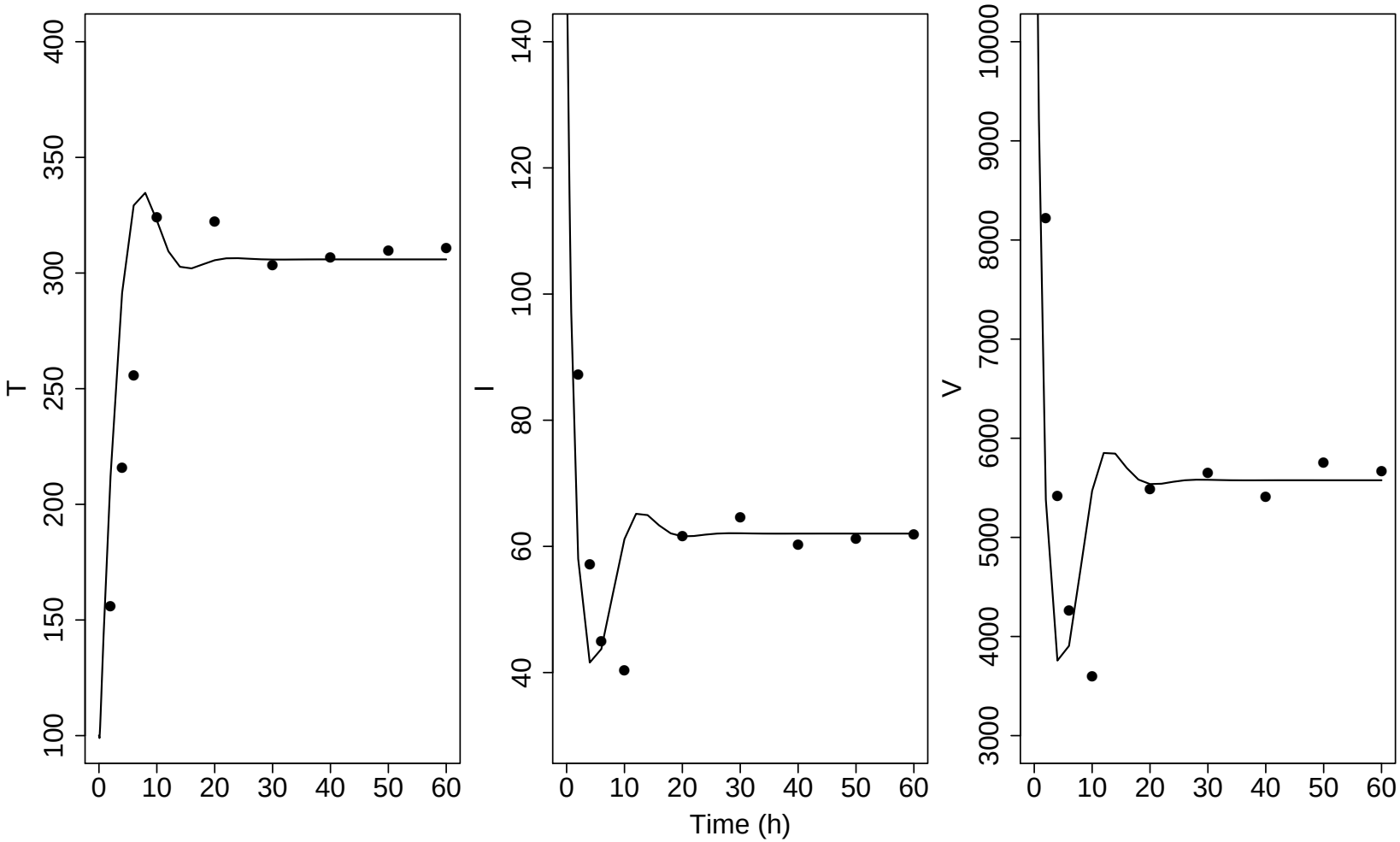
ODE model simulation - Jupyter languages

- Using DifferentialEquations.jl in Julia (Stan.jl)
- Using SciPy in Python (PySTAN)
- Using `deSolve` and `FME` packages in R (RSTAN)

Model output and data

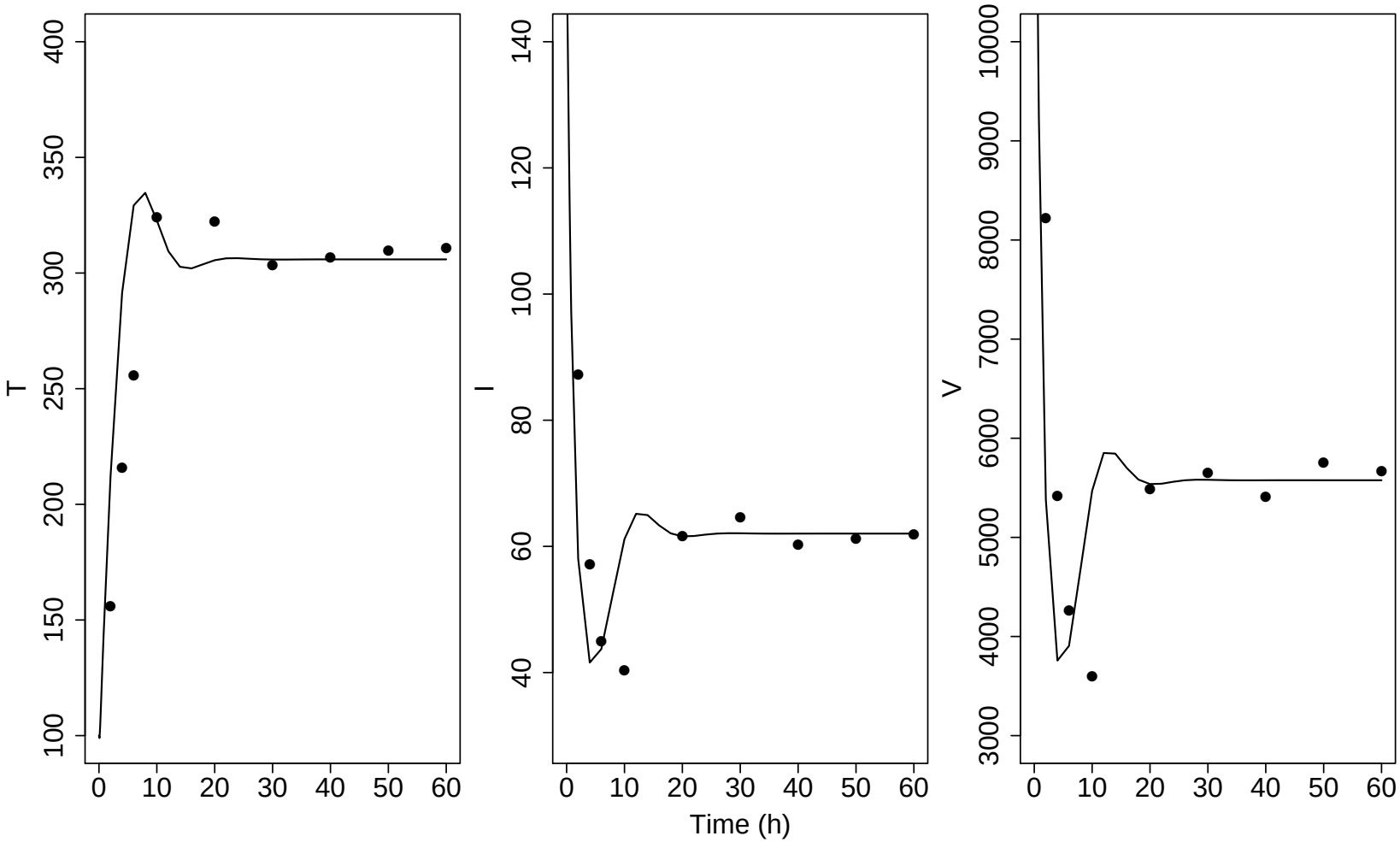


Model output and data



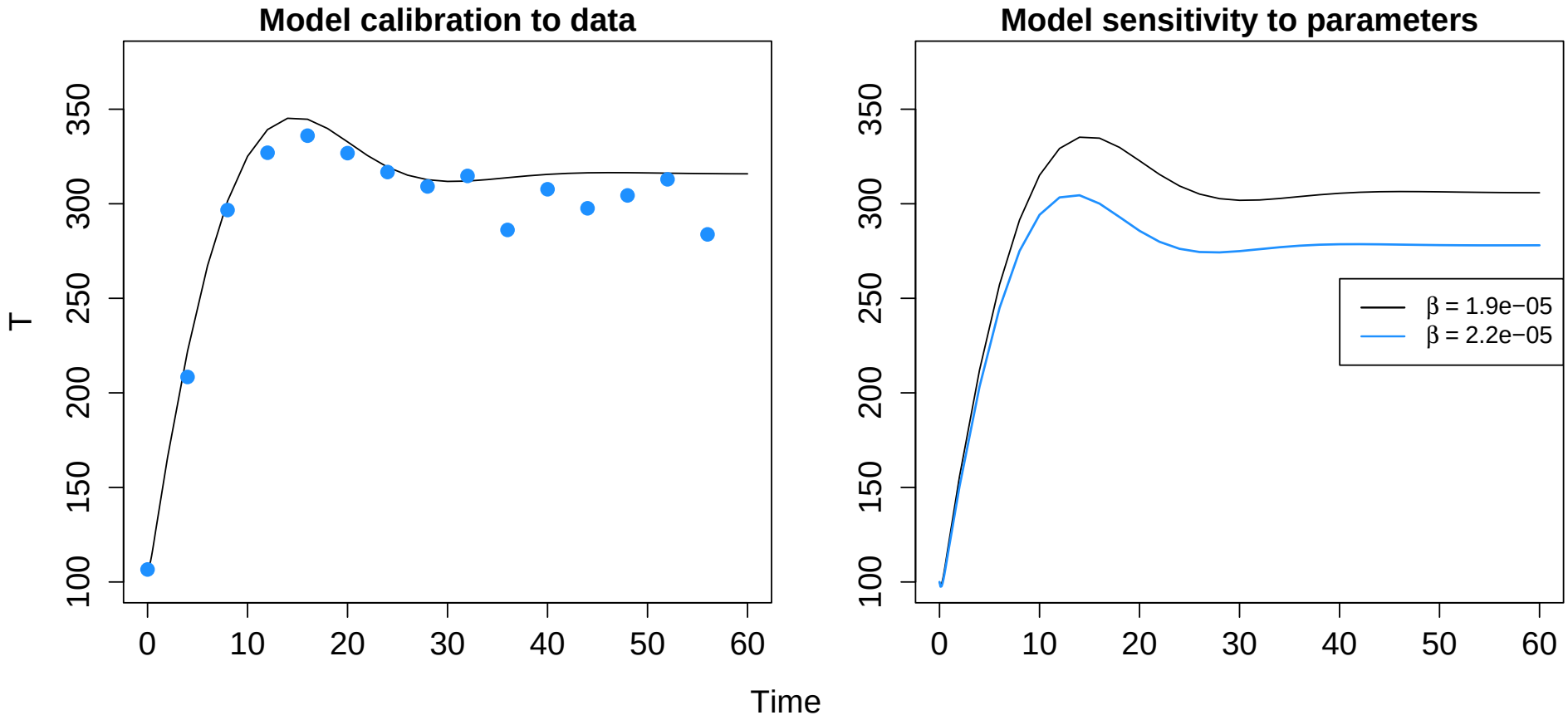
- Calibrate $\lambda, \rho, \beta, \delta, c$ and n

Model output and data



- Calibrate $\lambda, \rho, \beta, \delta, c$ and n
- T cell depletion/recovery and virion decay rates and (re)setpoints are relevant immunological characteristics

Data and model output and parameters



- Jointly identifiable parameters: distinct effects on model outputs
- Correlated parameters (non-jointly-identifiable) have not
- Identifiability analysis: include $\lambda, \rho, \beta, \delta, c$, exclude n

Model calibration 1: optimize the objective function

- Sum of squared residuals (SSR) for i timepoints and j output variables is often minimized:

$$SSR(\theta) = \sum_{i=1}^{n}\sum_{j=1}^{k}\left(\frac{\hat{y}_{ij} - y_{ij}}{\sigma_j}\right)^2 = \sum_{i=1}^{n}\sum_{j=1}^{k}\left(\frac{f(\theta, t_i)_{ij} - y_{ij}}{\sigma_j}\right)^2 \quad (4)$$

- Parameters are considered point estimates

Model calibration 1: optimize the objective function

- Sum of squared residuals (SSR) for i timepoints and j output variables is often minimized:

$$SSR(\theta) = \sum_{i=1}^{n}\sum_{j=1}^{k}\left(\frac{\hat{y}_{ij} - y_{ij}}{\sigma_j}\right)^2 = \sum_{i=1}^{n}\sum_{j=1}^{k}\left(\frac{f(\theta, t_i)_{ij} - y_{ij}}{\sigma_j}\right)^2 \quad (4)$$

- Parameters are considered point estimates
- Bayesian optimization takes parameters as random variables and solidly quantifies uncertainties

Model calibration 2: apply Bayes' theorem

For parameters that follow continuous distributions:

$$\overbrace{\underbrace{p(\boldsymbol{\theta}|\boldsymbol{y})}_{\text{posterior}} = \frac{p(\boldsymbol{\theta})L(\boldsymbol{\theta}|\boldsymbol{y})}{p(\boldsymbol{y})}}^{\text{Bayes' rule}} = \frac{p(\boldsymbol{\theta})L(\boldsymbol{\theta}|\boldsymbol{y})}{\int p(\boldsymbol{\theta})L(\boldsymbol{\theta}|\boldsymbol{y})d\boldsymbol{\theta}} \propto \underbrace{p(\boldsymbol{\theta})L(\boldsymbol{\theta}|\boldsymbol{y})}_{\text{prior times likelihood}} \tag{5}$$

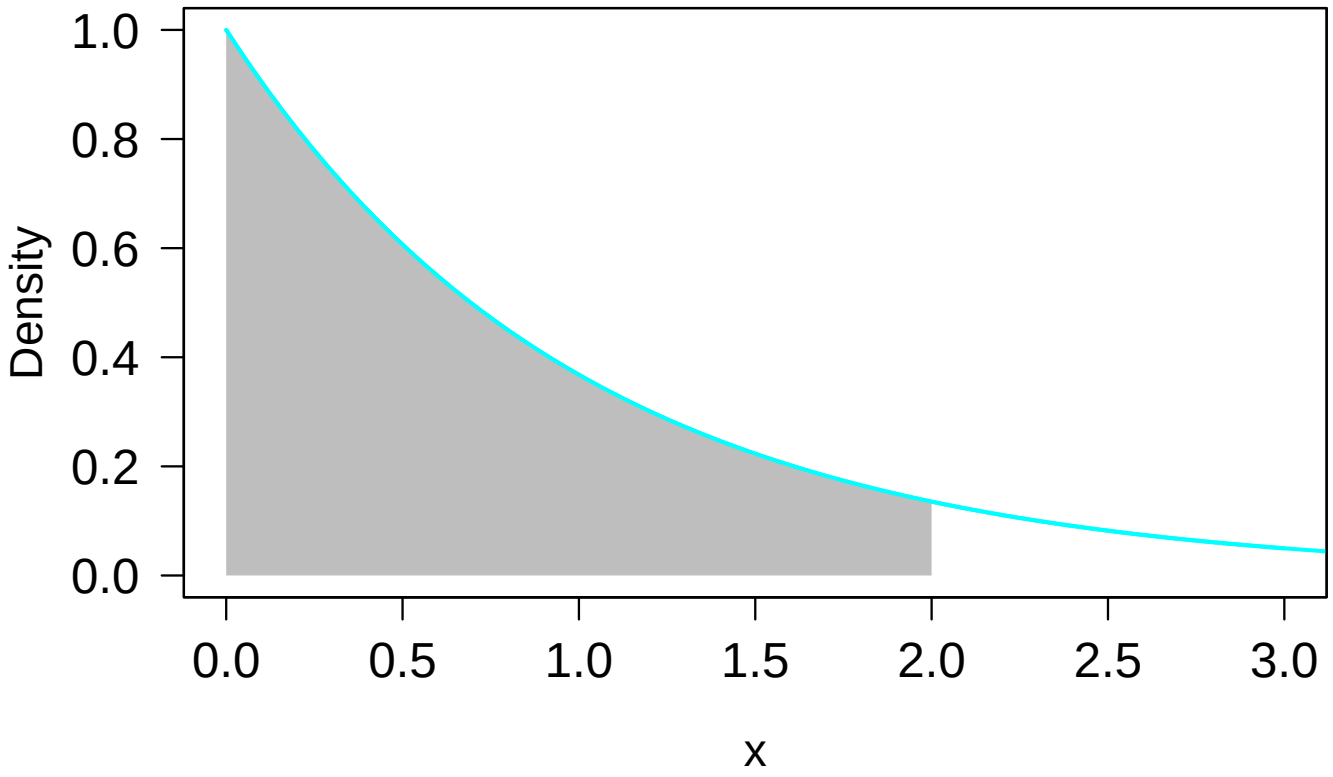
Model calibration 2: apply Bayes' theorem

For parameters that follow continuous distributions:

$$\overbrace{\underbrace{p(\boldsymbol{\theta}|\boldsymbol{y})}_{\text{posterior}} = \frac{p(\boldsymbol{\theta})L(\boldsymbol{\theta}|\boldsymbol{y})}{p(\boldsymbol{y})}}^{\text{Bayes' rule}} = \frac{p(\boldsymbol{\theta})L(\boldsymbol{\theta}|\boldsymbol{y})}{\int p(\boldsymbol{\theta})L(\boldsymbol{\theta}|\boldsymbol{y})d\boldsymbol{\theta}} \propto \underbrace{p(\boldsymbol{\theta})L(\boldsymbol{\theta}|\boldsymbol{y})}_{\text{prior times likelihood}} \quad (5)$$

- Parameters are considered random variables that may follow any distribution
- Obtain posterior knowledge through optimizing $L(\boldsymbol{\theta}|\boldsymbol{y})$ or $p(\boldsymbol{\theta})L(\boldsymbol{\theta}|\boldsymbol{y})$, i.e. use data and prior knowledge
- Various Markov-chain Monte Carlo algorithms are used for optimization in Bayesian packages

Monte Carlo integration of an exponential distribution



$$\int_0^2 \lambda \cdot \exp^{-\lambda \cdot x} dx$$

```
# Take a random sample of x values on [0,2]
x <- sort(runif(10000, 0, 2))

# Perform the numerical integration
cat("Integral value is equal to:", 2*mean(dexp(x, rate=1)), "\n")
```

```
## Integral value is equal to: 0.8622298
```

Markov-chain Monte Carlo: Metropolis-Hastings and Gibbs

Metropolis-Hastings updates of the Markov chain

	$P(\theta_1\|y) > P(\theta_0\|y)$	$P(\theta_1\|y) < P(\theta_0\|y)$
$\theta_0 \to \theta_1$	θ_1 always adopted	θ_1 adopted less likely with decreasing $\pi(\theta_1\|y) < \pi(\theta_0\|y)$

Markov-chain Monte Carlo: Metropolis-Hastings and Gibbs

Metropolis-Hastings updates of the Markov chain

	$P(\theta_1\|y) > P(\theta_0\|y)$	$P(\theta_1\|y) < P(\theta_0\|y)$
$\theta_0 \to \theta_1$	θ_1 always adopted	θ_1 adopted less likely with decreasing $\pi(\theta_1\|y) < \pi(\theta_0\|y)$

Gibbs sampling - Implemented in WinBUGS, MCMCglmm, etc

- If the conditional distribution is known, e.g. for $P(\mu|\sigma = 1, y)$, the next value can be sampled using that conditional distribution
- More targeted, but only one parameter updated at a time

Markov-chain Monte Carlo: Metropolis-Hastings and Gibbs

Metropolis-Hastings updates of the Markov chain

	$P(\theta_1\|y) > P(\theta_0\|y)$	$P(\theta_1\|y) < P(\theta_0\|y)$
$\theta_0 \to \theta_1$	θ_1 always adopted	θ_1 adopted less likely with decreasing $\pi(\theta_1\|y) < \pi(\theta_0\|y)$

Gibbs sampling - Implemented in WinBUGS, MCMCglmm, etc

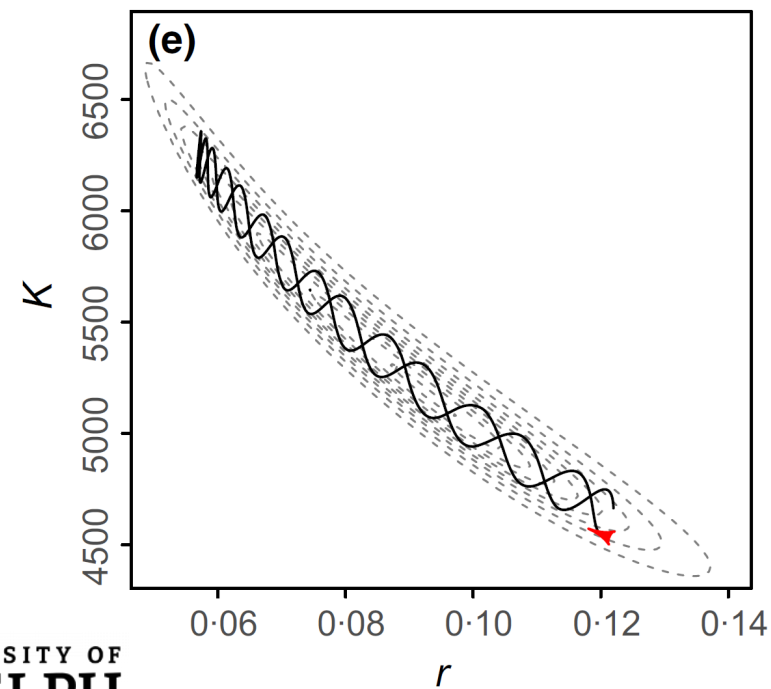
- If the conditional distribution is known, e.g. for $P(\mu|\sigma = 1, y)$, the next value can be sampled using that conditional distribution
- More targeted, but only one parameter updated at a time

⋆ These two samplers have been used for quite some time and are implemented in various packages

Markov-chain Monte Carlo: HMC

Hamiltonian Monte Carlo - implemented in STAN

- Uses gradients of the log posterior density with respect to parameter values to find high probability regions of the parameter space
- Multiple parameters updated at a time
- Faster than M-H and Gibbs for models with numerous parameters



Monnahan et al., (2017)

Bayesian model and prior knowledge

Model equation

$$y_{ij} = f(\boldsymbol{\theta}, t_i)_{ij} + e_{ij} \tag{6}$$

Bayesian model and prior knowledge

Model equation

$$y_{ij} = f(\boldsymbol{\theta}, t_i)_{ij} + e_{ij} \tag{6}$$

Priors phase 1 - data distributions

$$y_{ij} \sim MVN(f(\boldsymbol{\theta}, t_i), \Sigma) \tag{7}$$

$$\Sigma = \begin{bmatrix} \sigma_1 & 0 & 0 \\ 0 & \sigma_2 & 0 \\ 0 & 0 & \sigma_3 \end{bmatrix}$$

Bayesian model and prior knowledge

Model equation

$$y_{ij} = f(\boldsymbol{\theta}, t_i)_{ij} + e_{ij} \tag{6}$$

Priors phase 1 - data distributions

$$y_{ij} \sim MVN(f(\boldsymbol{\theta}, t_i), \Sigma) \tag{7}$$

$$\Sigma = \begin{bmatrix} \sigma_1 & 0 & 0 \\ 0 & \sigma_2 & 0 \\ 0 & 0 & \sigma_3 \end{bmatrix}$$

Priors phase 2 - parameter distributions

$$\boldsymbol{\theta} \sim N(\boldsymbol{\theta}_{start}, 0.25 \cdot \boldsymbol{\theta}_{start}) \tag{8}$$

$$\sigma_j \sim \text{half} - \text{Cauchy}(0, 5) \tag{9}$$

STAN code - 1

```
HIV_normal_mv <- "

functions {
  real[] HIV(real t, real[] y, real[] theta,
             real[] x_r, int[] x_i){

    // Initial values
    real T = y[1]; real I = y[2]; real V = y[3];

    // Model parameters
    real bet = theta[1]; real rho = theta[2]; real delt = theta[3];
    real c = theta[4]; real lam = theta[5];
    real n = 900.0;

    // Differential equations
    real dT_dt = lam - rho * T - bet * T * V;
    real dI_dt = bet * T * V - delt * I;
    real dV_dt = n * delt * I - c * V - bet * T * V;

    return{dT_dt, dI_dt, dV_dt};
  } // End of HIV function
} // End functions block
 ...
```

STAN code - 2

```
data{
  int<lower=1> n_obs; // number of times sampled
  int<lower=1> n_diffeq; // number of differentials of system
  real y0[n_diffeq]; // Initial values of state variables
  real t0; // Initial time point (zero)
  real ts[n_obs]; // time samples of observations
  row_vector[n_diffeq] y[n_obs]; // observed concentrations
} // End data block


transformed data {
  real x_r[0];
  int x_i[0];
} // End tf data block


parameters{
  real<lower=0> theta[5];
  vector<lower=0>[n_diffeq] sigma_e;  // init residual SD matrix
} // End parameters block
 ...
```

STAN code - 3

```
transformed parameters{
  real y_hat[n_obs, n_diffeq]; // Output from ODE solver
  row_vector[n_diffeq] mu[n_obs]; // vector/matrix for ODE output

  y_hat = integrate_ode_rk45(HIV, y0, t0, ts, theta, x_r, x_i); // solve ODE model
  for (i in 1:n_obs) mu[i] = to_matrix(y_hat)[i,]; // transform output from real to row_vect
} // End tf parameters block

model {
  // priors
  theta[1] ~ normal(4e-5, 2e-5);
  theta[2] ~ normal(0.3, 0.15);
  theta[3] ~ normal(1, 0.5);
  theta[4] ~ normal(11, 5);
  theta[5] ~ normal(100, 10);
  sigma_e ~ cauchy(0, 5);

  // Likelihood/sampling distribution
  y ~ multi_normal(mu, diag_matrix(sigma_e) ); // likelihood
} // End model block

generated quantities {
  vector[n_diffeq] pred_y[n_obs];
  pred_y = multi_normal_rng(mu, diag_matrix(sigma_e)); // Posterior predictive distribution
} // End generated quantities
"
```

RSTAN code - Input - Output

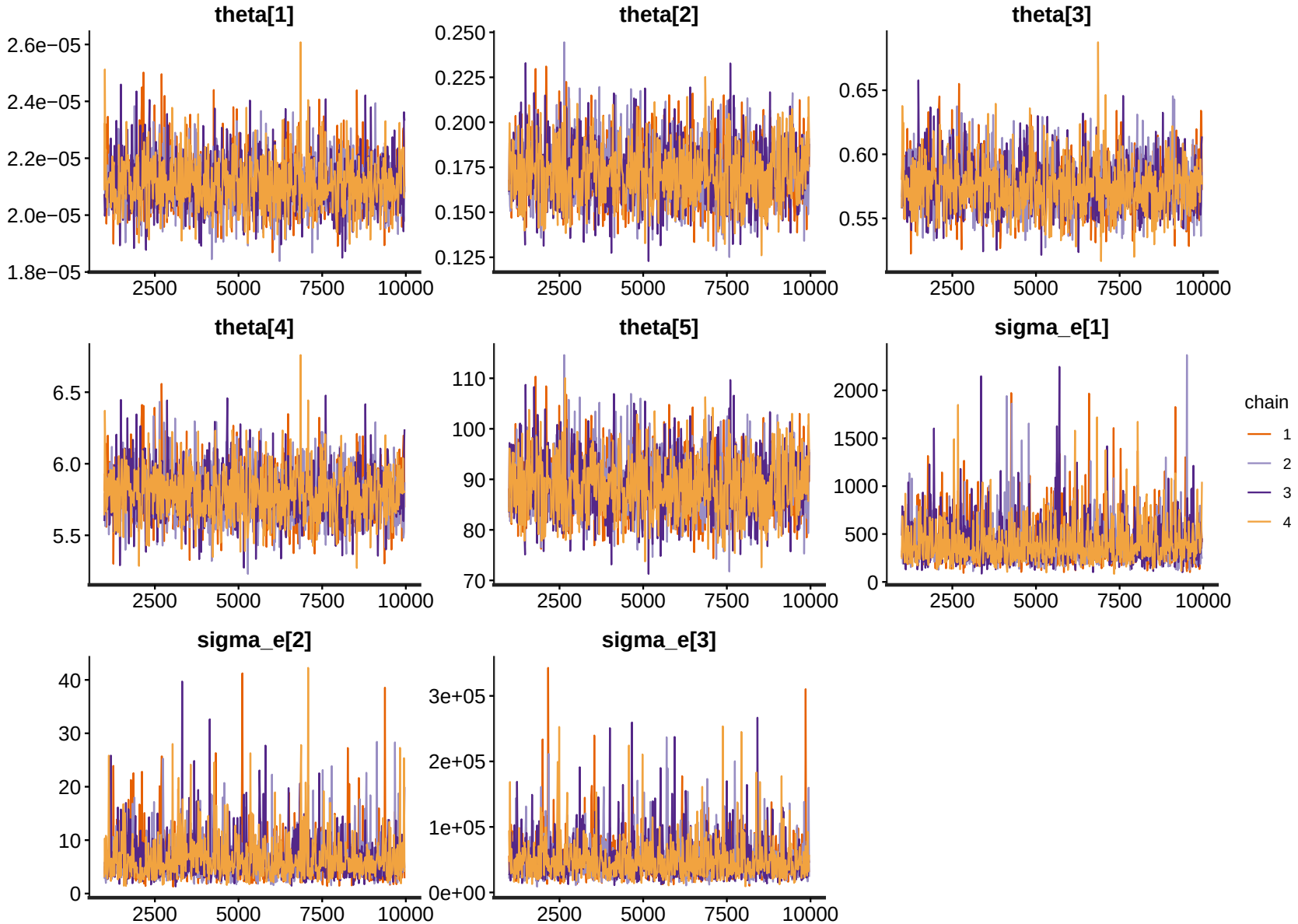
```
library(rstan)
rstan_options(auto_write=TRUE)
options(mc.cores=parallell::detectCores())

HIV_model_mv <- stan_model(model_code=HIV_normal_mv, model_name="HIV_normal_mv")

y_0 <- c(T=100, I=160, V=10e4)
t_s <- c(2, 4, 6, seq(10,60,10))
X_s <- X[,c('T', 'I', 'V')]

data_HIV_mv <- list(n_obs=9, n_diffeq=3, y0=y_0, t0=0, ts=t_s, y=X_s)

fit_HIV_mv <- sampling(HIV_normal_mv, data=data_HIV_mv,
                       iter=52e3, warmup=2e3, thin=15, chains=4)
```

RSTAN code - Input - Output

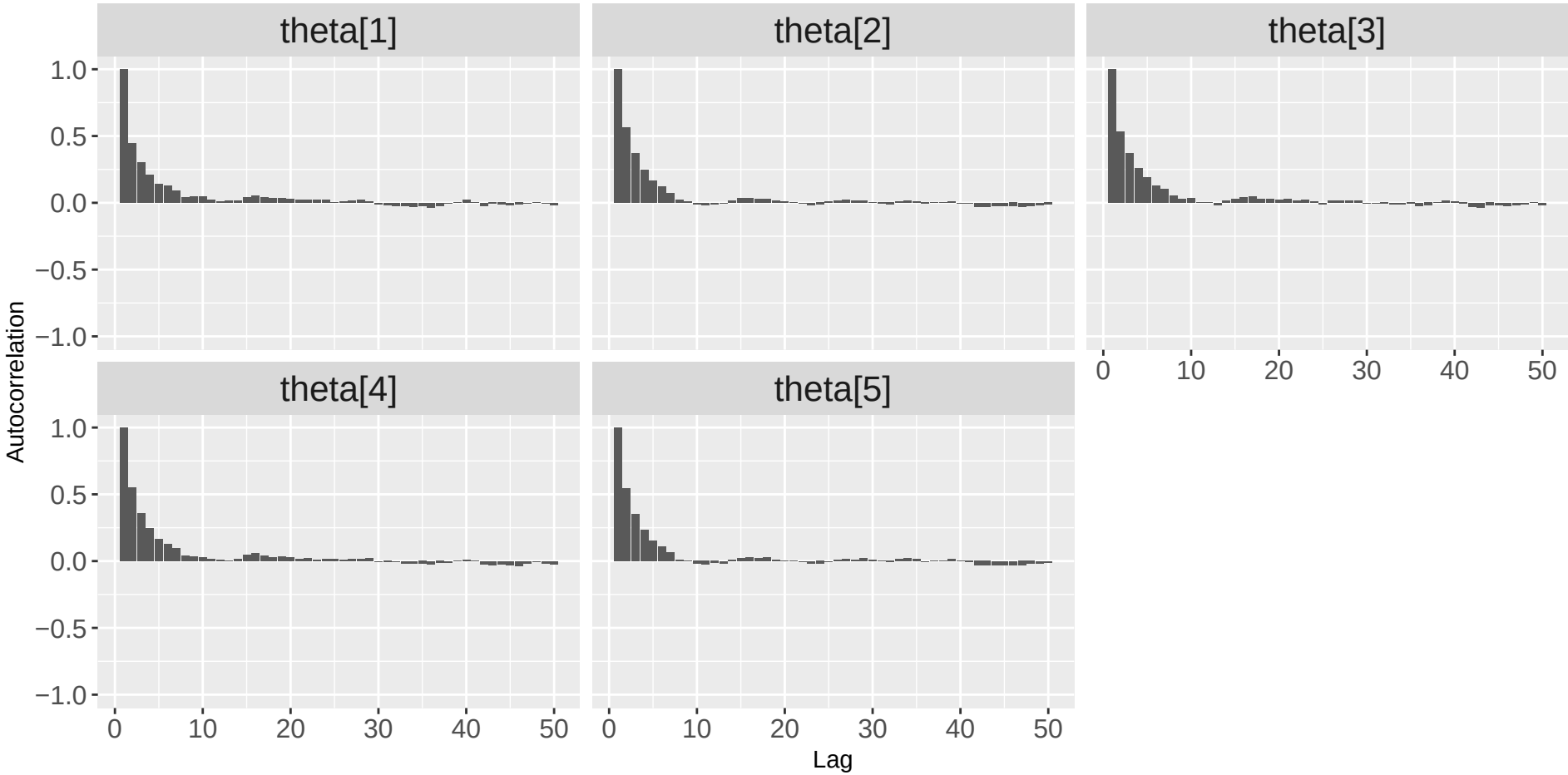
```
library(rstan)
rstan_options(auto_write=TRUE)
options(mc.cores=parallell::detectCores())

HIV_model_mv <- stan_model(model_code=HIV_normal_mv, model_name="HIV_normal_mv")

y_0 <- c(T=100, I=160, V=10e4)
t_s <- c(2, 4, 6, seq(10,60,10))
X_s <- X[,c('T', 'I', 'V')]

data_HIV_mv <- list(n_obs=9, n_diffeq=3, y0=y_0, t0=0, ts=t_s, y=X_s)

fit_HIV_mv <- sampling(HIV_normal_mv, data=data_HIV_mv,
                       iter=52e3, warmup=2e3, thin=15, chains=4)
```

```
> summary(fit_HIV_mv, pars=c("theta","sigma_e","pred_y")$summary

                mean   se_mean         sd       2.5%        25%        50%        75%      97.5% n_eff  Rhat
theta[1]    2.071e-05 7.813e-09 8.833e-07  1.910e-05  2.011e-05 2.067e-05  2.126e-05  2.258e-05 12780 1.000
theta[2]    1.514e-01 1.279e-04 1.460e-02  1.245e-01  1.416e-01 1.506e-01  1.604e-01  1.827e-01 13028 1.000
theta[3]    5.565e-01 1.664e-04 1.834e-02  5.214e-01  5.447e-01 5.562e-01  5.679e-01  5.945e-01 12139 1.000
 ...
sigma_e[1]  2.192e+02 1.143e+00 1.297e+02  8.465e+01  1.392e+02 1.867e+02  2.591e+02  5.447e+02 12875 1.000
sigma_e[2]  8.867e+00 5.526e-02 6.387e+00  2.528e+00  4.854e+00 7.175e+00  1.075e+01  2.510e+01 13357 1.000
 ...
pred_y[1,1] 1.255e+02 1.285e-01 1.495e+01  9.511e+01  1.164e+02 1.256e+02  1.349e+02  1.554e+02 13533 1.000
pred_y[1,2] 1.154e+02 2.820e-02 3.210e+00  1.091e+02  1.135e+02 1.153e+02  1.173e+02  1.219e+02 12951 1.000
 ...
```

Model parameter chains for assessing convergence

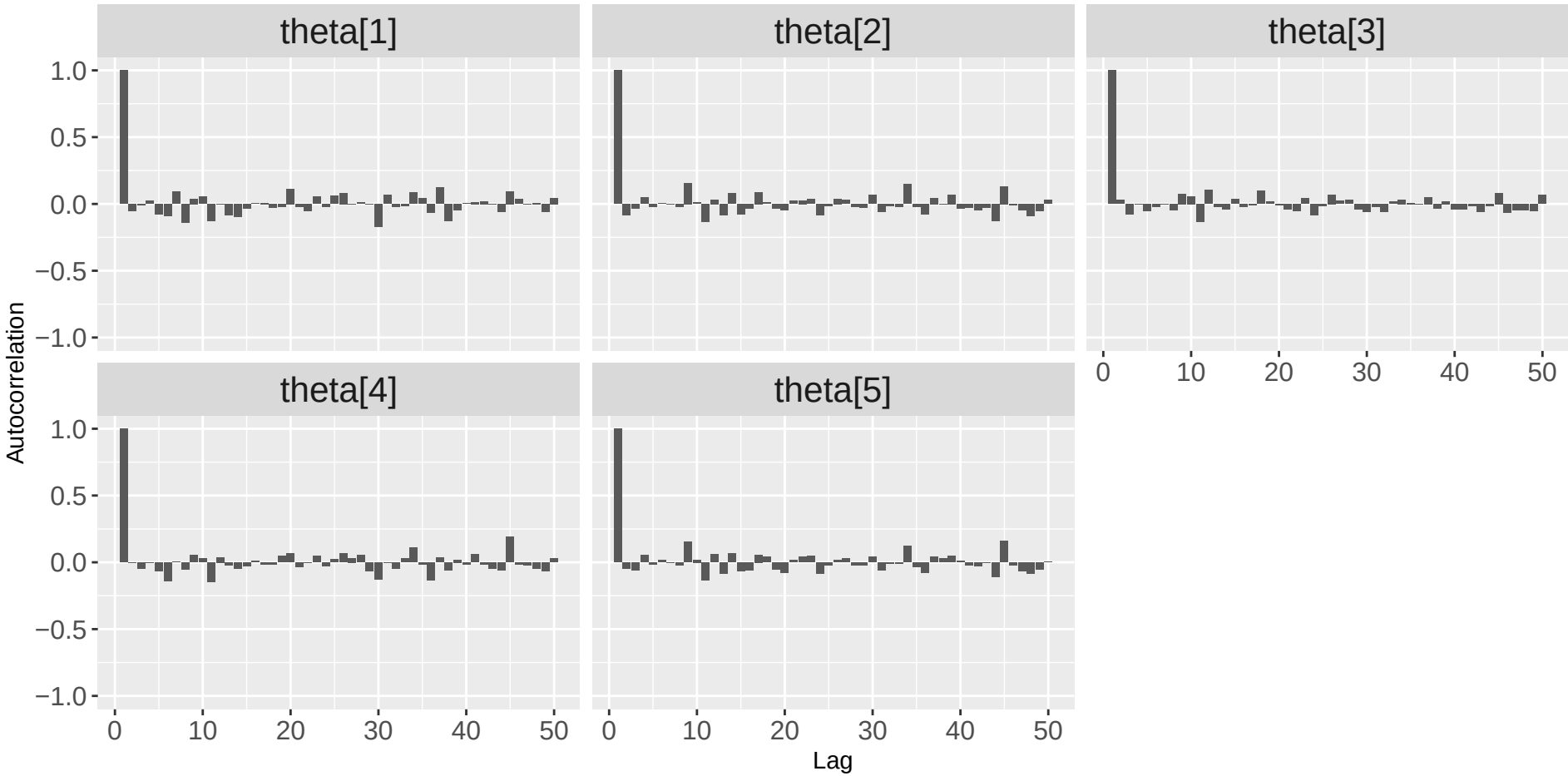


Autocorrelation plots - Thinning = 1



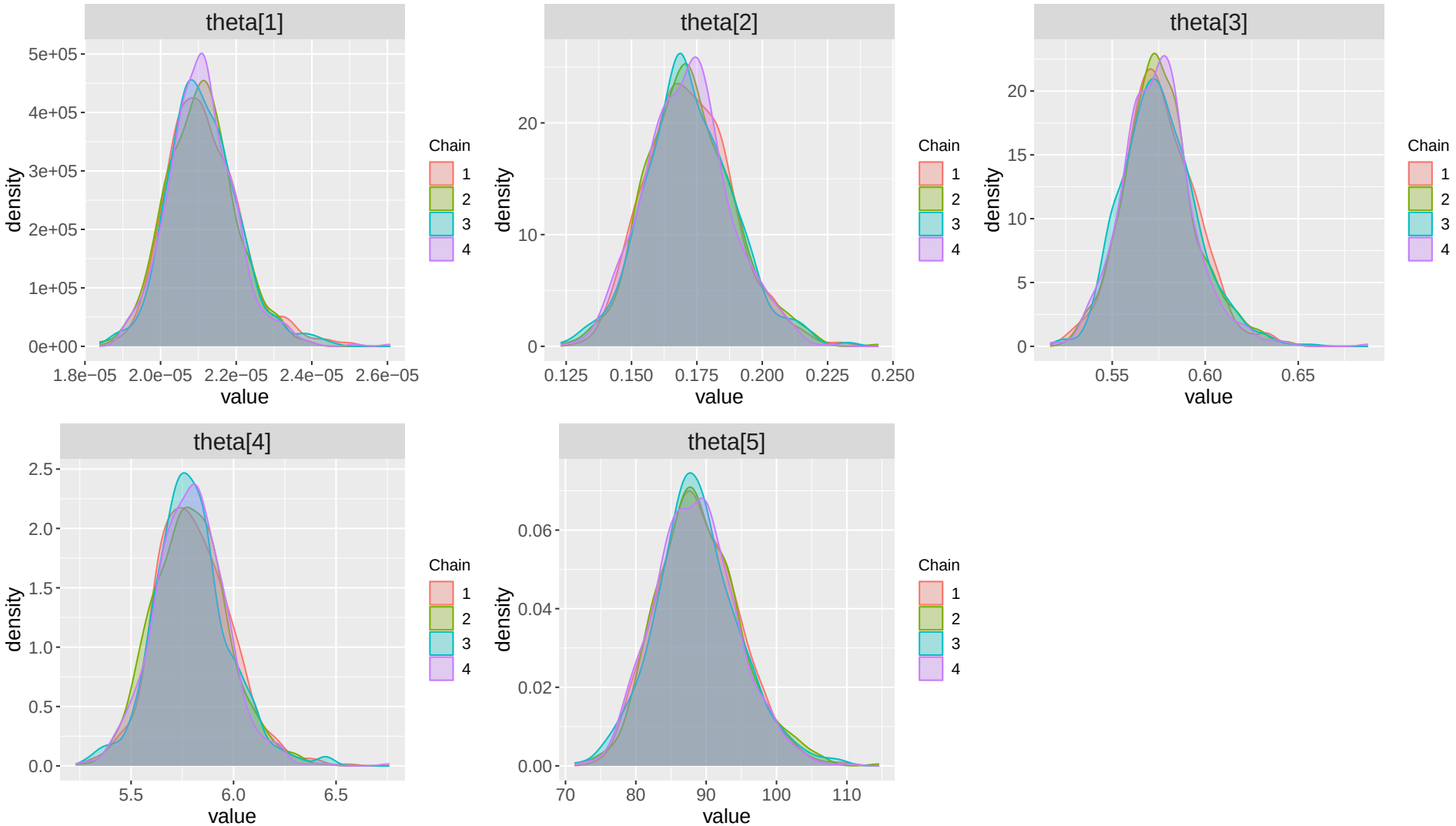
- Some serial correlation

Autocorrelation plots - Thinning = 15



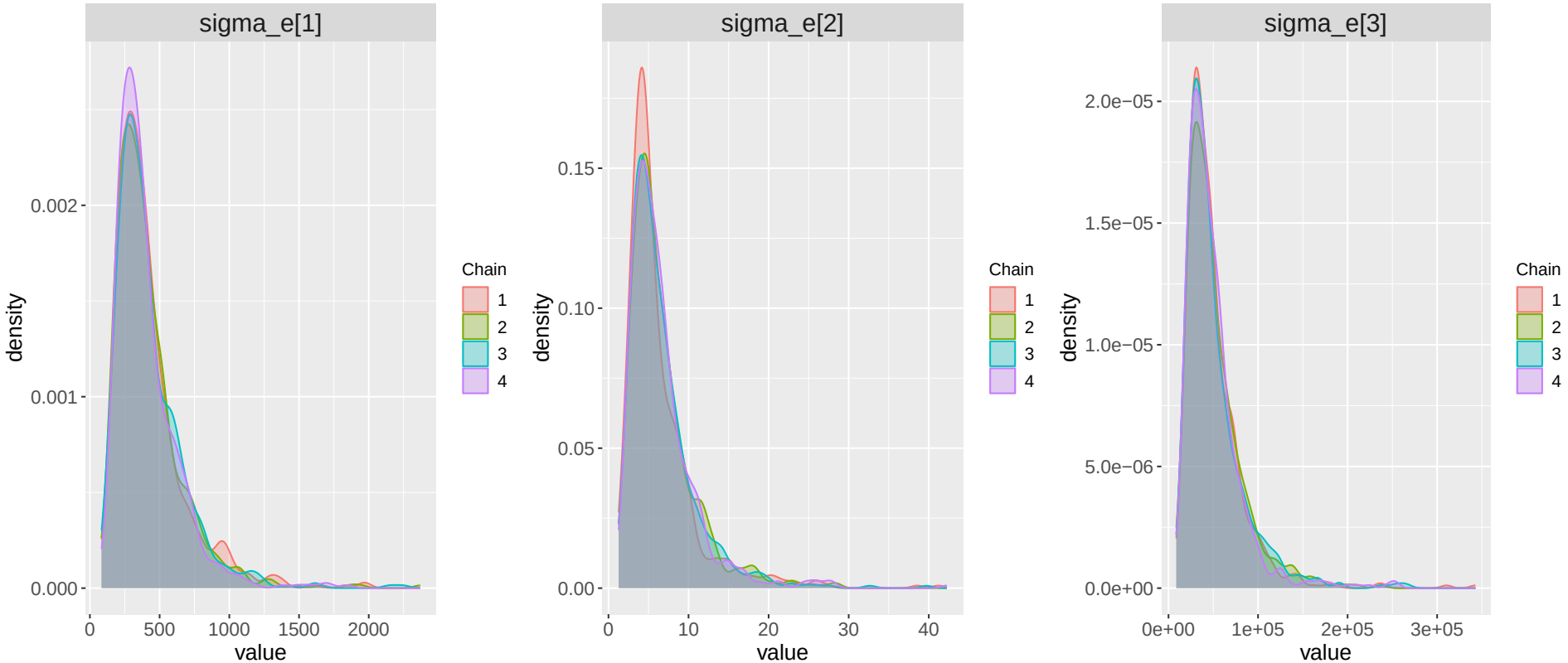
- Limited and more random autocorrelation patterns

Model parameter distributions of θ



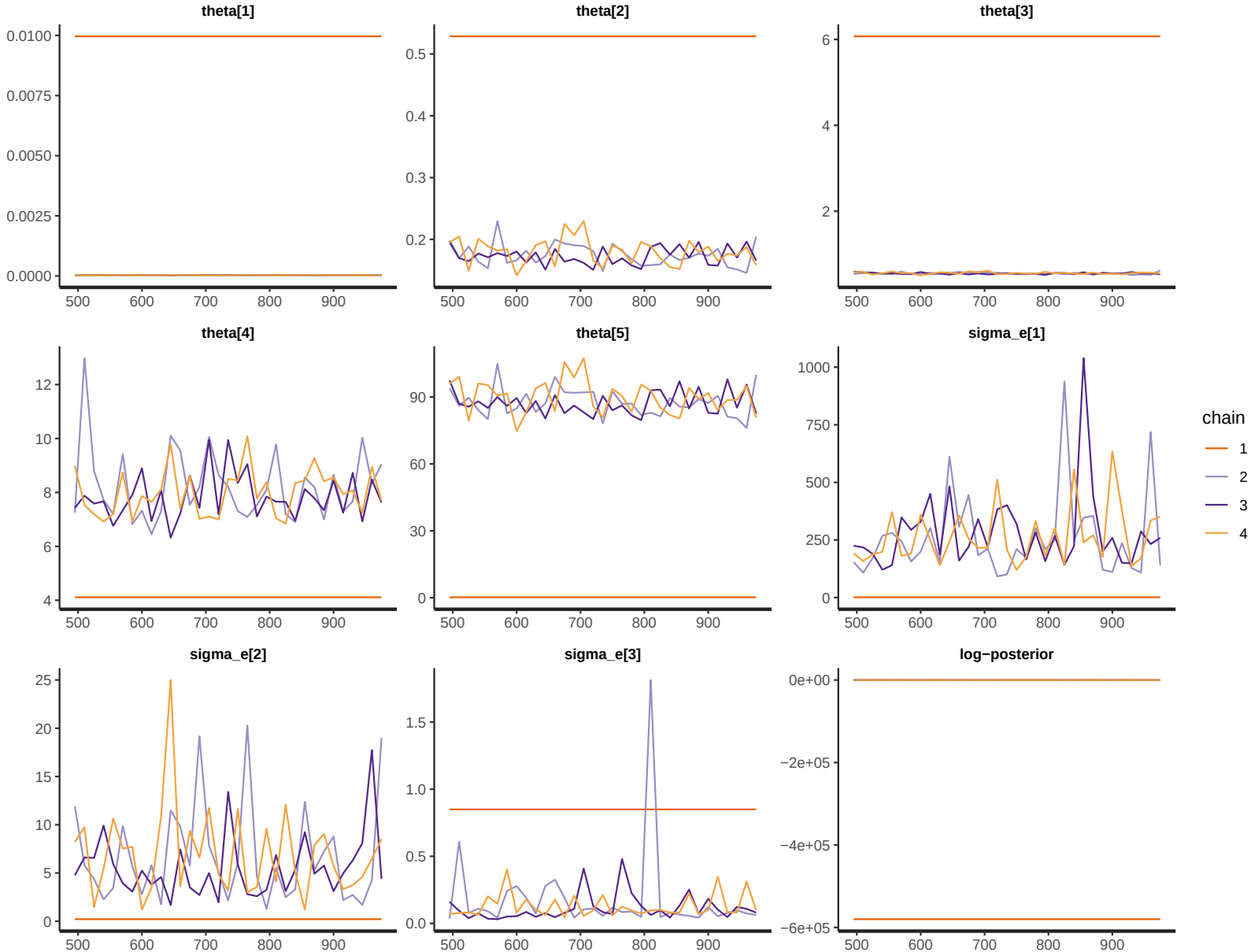
- All θ different from zero

Model parameter distributions of σ



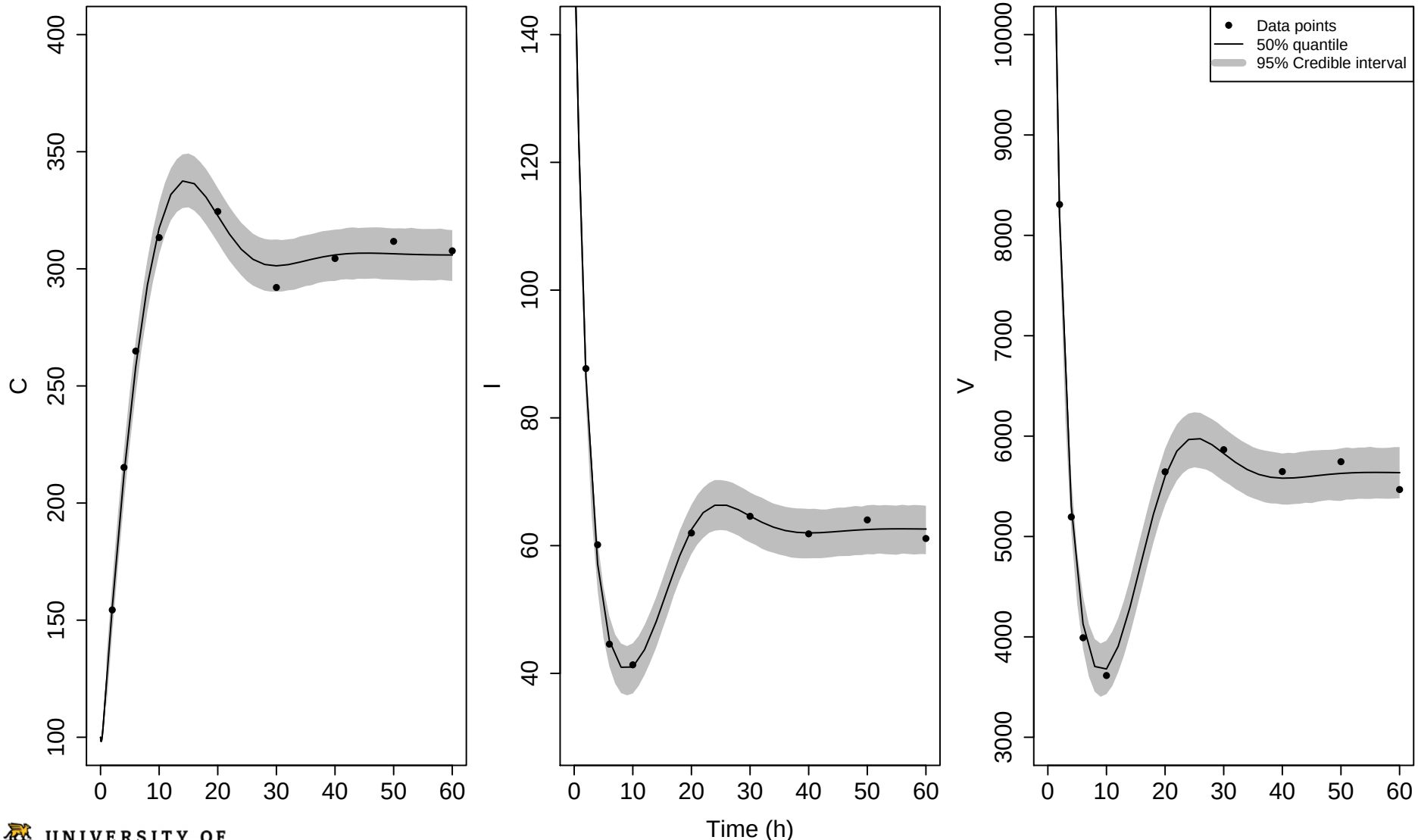
- σ_e's relatively close to zero and tailed to the right
- Re-evaluate σ_e's for noisier data

Reconsider the approach ... ?



Posterior predictive distributions

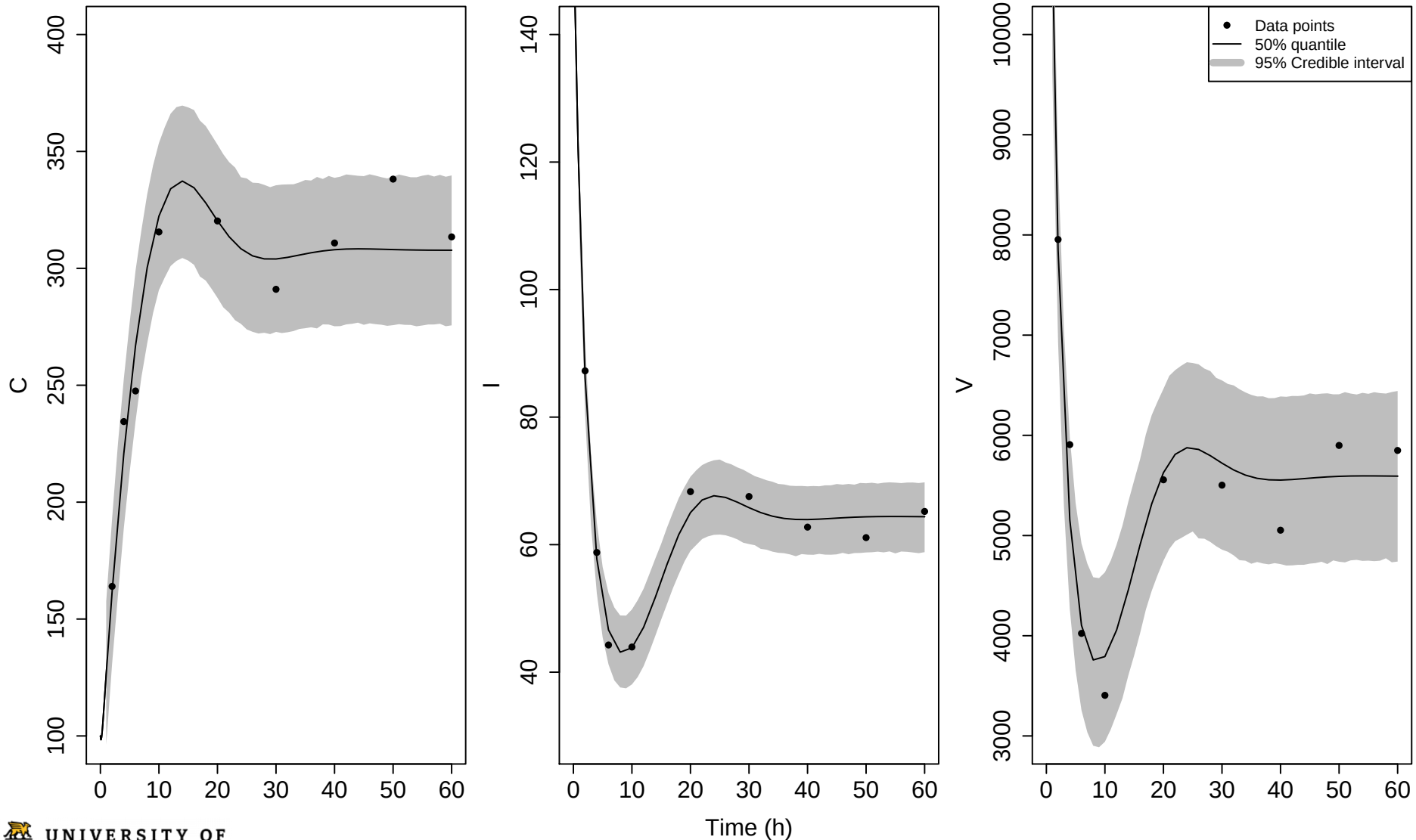
Data with *minor* noise and *informative* priors used



UNIVERSITY OF GUELPH

Posterior predictive distributions

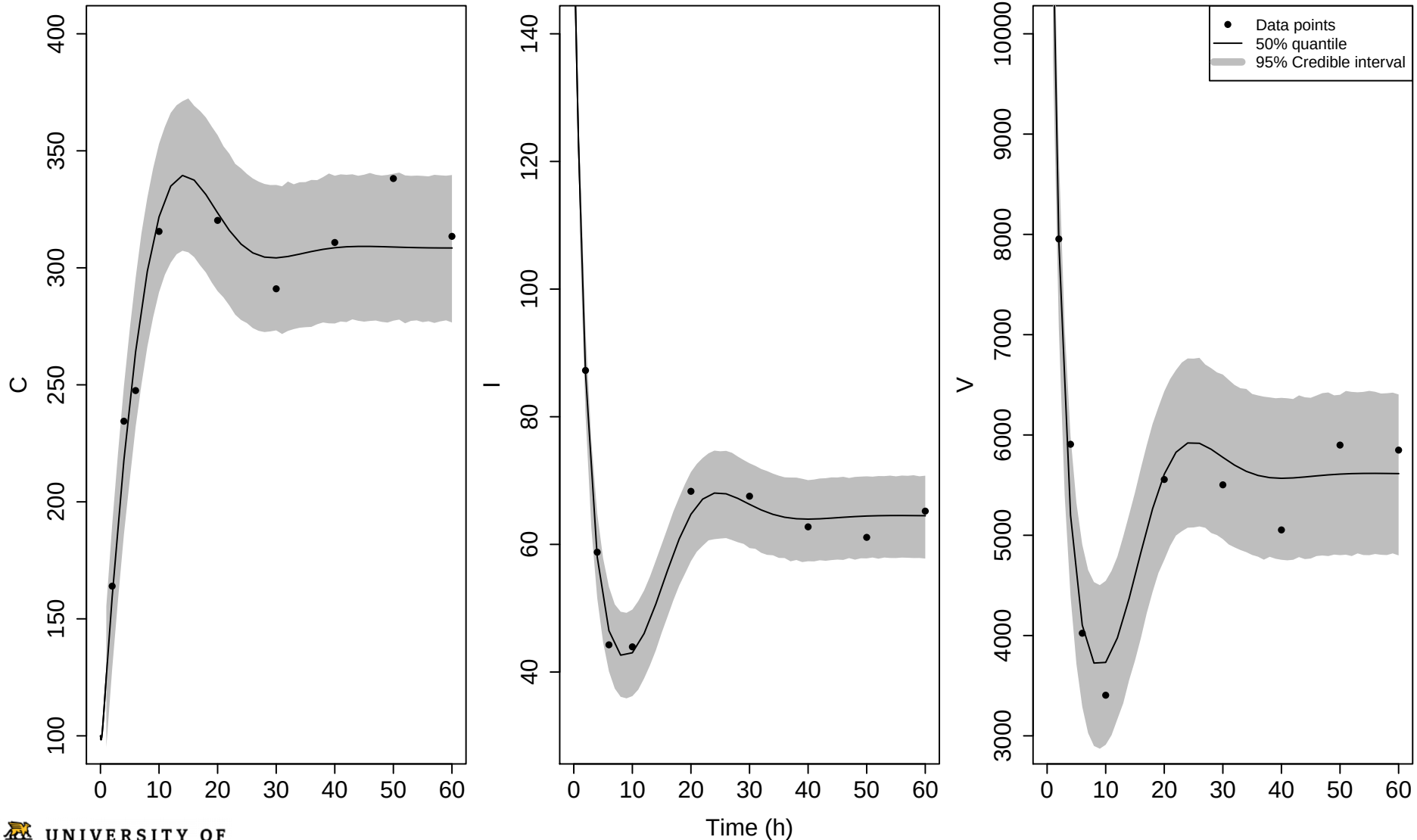
Data with *moderate* noise and *informative* priors used



UNIVERSITY OF GUELPH

Posterior predictive distributions

Data with *moderate* noise and *weakly informative* priors used



UNIVERSITY OF GUELPH

Consider the SIR/SEIR model

$$\frac{dS}{dt} = \frac{-\beta \cdot I \cdot S}{N} \quad \text{[susceptible people]} \tag{10}$$

$$\frac{dI}{dt} = \frac{\beta \cdot I \cdot S}{N} - \gamma \cdot I \quad \text{[infected people]} \tag{12}$$

$$\frac{dR}{dt} = \gamma \cdot I \quad \text{[recovered people]} \tag{13}$$

$$R_0 = \beta/\gamma \quad \text{[reproduction number]} \tag{14}$$

- Transmission rate β and recovery rate γ

Consider the SIR/SEIR model

$$\frac{dS}{dt} = \frac{-\beta \cdot I \cdot S}{N} \quad \text{[susceptible people]} \tag{10}$$

$$\frac{dE}{dt} = \ldots \quad \text{[exposed people; e.g. Andrade \& Duggan, 2021]} \tag{11}$$

$$\frac{dI}{dt} = \frac{\beta \cdot I \cdot S}{N} - \gamma \cdot I \quad \text{[infected people]} \tag{12}$$

$$\frac{dR}{dt} = \gamma \cdot I \quad \text{[recovered people]} \tag{13}$$

$$R_0 = \beta/\gamma \quad \text{[reproduction number]} \tag{14}$$

- Transmission rate β and recovery rate γ

Conclusion

- Bayesian optimization takes parameters as random variables, which solidly quantifies uncertainty

Conclusion

- Bayesian optimization takes parameters as random variables, which solidly quantifies uncertainty
- Prior knowledge and data generate posterior knowledge (i.e. revised prior knowledge)

Conclusion

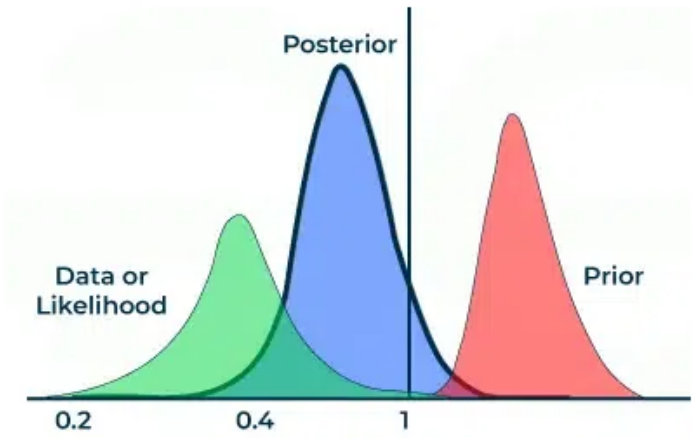
- Bayesian optimization takes parameters as random variables, which solidly quantifies uncertainty
- Prior knowledge and data generate posterior knowledge (i.e. revised prior knowledge)
- Hamiltonian Monte Carlo powerfully explores prior $\times$ likelihood and samples from the posterior

Conclusion

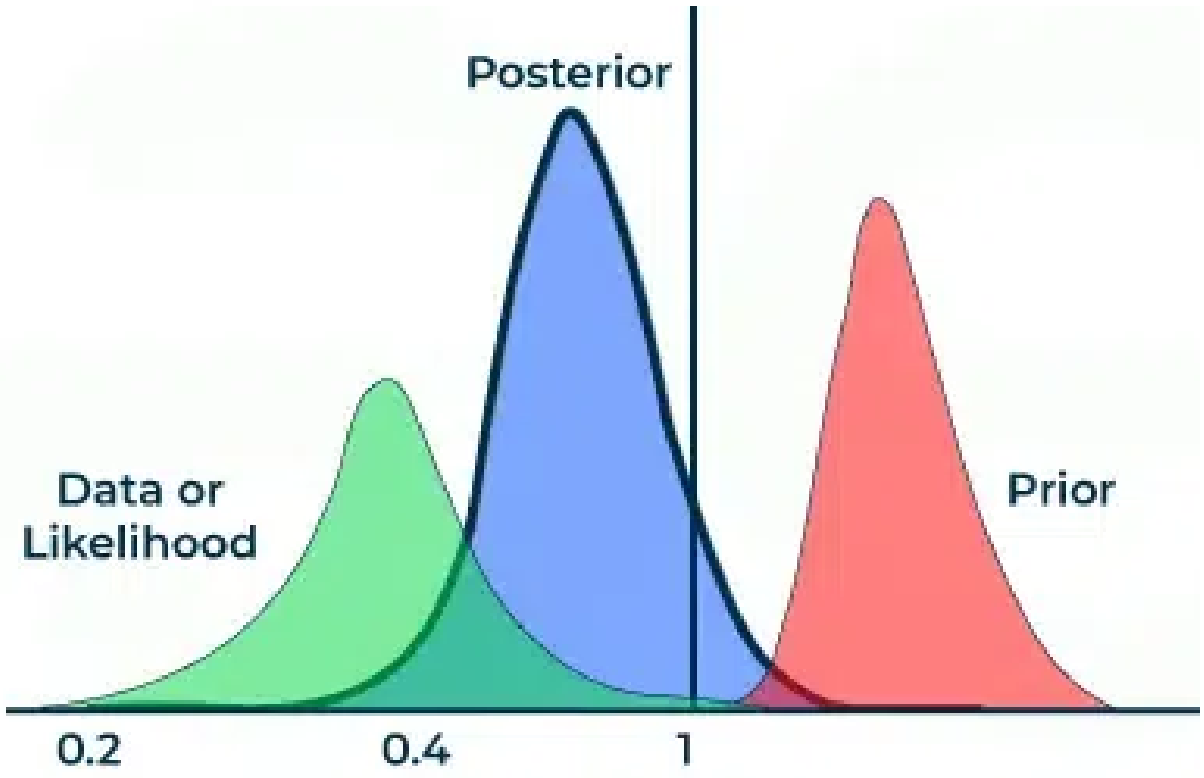
- Bayesian optimization takes parameters as random variables, which solidly quantifies uncertainty
- Prior knowledge and data generate posterior knowledge (i.e. revised prior knowledge)
- Hamiltonian Monte Carlo powerfully explores prior $\times$ likelihood and samples from the posterior
- Posterior predictive distributions a) visualize if the model fits the data well and b) help to test hypotheses

The end of our course, what's next?

- You learned the basics of Bayesian statistics applied to various statistical approaches
- I encourage you to continue using Bayesian methods and analyze your own data
- Note the STAN reference manual: `https://mc-stan.org/docs/`
- You can always reach out to me for discussion and guidance
- Last but not least, make sure you enjoy your Bayesian data analyses!



Thank you very much for your attention this week!



Questions?

Exercise lecture 6

- Reproduce parameter optimization procedure
- Simulate data with greater noise and re-perform parameter optimization
- Consider various informative and non-informative priors you did not use before
- Perform a complete parameter calibration for the SIR/SEIR model (Grinsztajn et al., 2019; Andrade & Duggan, 2021) model starting with the R code (this includes data through a package) on GitHub